

Intrusiveness and Scalability of OMPT-Based Tracing Tools for Task-based OpenMP Applications

Rayan Raddatz de Matos, Lucas Mello Schnorr

{rayan.raddatz@inf.ufrgs.br, schnorr@inf.ufrgs.br}

Agenda

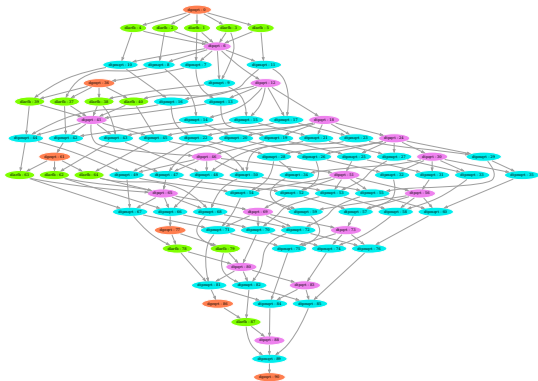
- Task-based OpenMP Applications
- OMPT-Based Tracing Tools
- Intrusiveness and Scalability

Intrusiveness and Scalability of OMPT-Based Tracing Tools for Task-based OpenMP Applications

Intrusiveness and Scalability of OMPT-Based Tracing Tools for

Task-based OpenMP Applications

Task-based parallelism

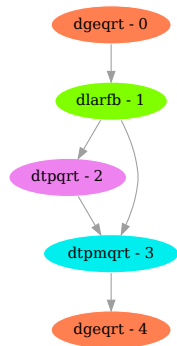


Programmer: Define the tasks and its parameters.

Runtime system: Schedule tasks to the computational cores.

Task-based parallelism

Can be viewed as a TDG
(Task Dependency Graph)



- Shared-memory multiprocessing programming interface.
- Since 3.0 supports tasks.
- New features for tasks with newer versions.



```
int main(){  
    #pragma omp parallel  
    {  
        #pragma omp single  
        {  
            for(int i = 0; i < 10; i++){  
                #pragma omp task  
                compute();  
            }  
        }  
    }  
}
```


- Application behavior is not simple to understand.

- Application behavior is not simple to understand.
- Need for tracing tools capable of capture tasks life cycle.

Intrusiveness and Scalability of OMPT-Based Tracing Tools for Task-based OpenMP Applications

Introduced in OpenMP 5.

User-defined callbacks when
OpenMP events occurs.

OMPT: An OpenMP Tools Application Programming Interface for Performance Analysis

Alexandre E. Eichenberger¹, John Mellor-Crummey², Martin Schulz³,
Michael Wong⁴, Nawal Copt⁵, Robert Dietrich⁶, Xu Liu², Eugene Loh⁷,
and Daniel Lorenz⁸
and other members of the OpenMP Tools Working Group

¹ IBM T.J. Watson Research Center

² Rice University

³ LLNL

⁴ IBM Canada

⁵ Oracle

⁶ TU Dresden, ZIH

⁷ Juelich Supercomputer Center

- Additional code to trace application behavior.

- Additional code to trace application behavior.
- Changes the application environment and **introduce overhead.**

Our tools

- Void
- Printf
- OOT

Popular tools

- Score-P
- TiKKi
- Extrae

```
static void on_ompt_callback_task_create(  
    ompt_data_t *parent_task_data,  
    const ompt_frame_t *parent_frame,  
    ompt_data_t* new_task_data,  
    int flag,  
    int has_dependences,  
    const void *codeptr_ra)  
{  
    /* Nothing here */  
}
```



```
static void on_ompt_callback_task_create(
    omp_data_t *parent_task_data,
    const omp_frame_t *parent_frame,
    omp_data_t* new_task_data,
    int flag,
    int has_dependences,
    const void *codeptr_ra)
{
    new_task_data->value = next_task();

    int thread_id = omp_get_thread_num();
    if (!arqs[thread_id]) {
        fprintf(stderr, "Tracefile for this thread is not open for writing.\n");
        exit(1);
    }

    if (flag != omp_task_initial) {
        fprintf(arqs[thread_id], "%d %ld task_create %s %lf %ld\n", omp_get_thread_num(), new_task_data->value,
            TASKS[ACTUAL_TASK], omp_get_wtime()-INIT_TIME, ITERATION);
    } else {
        fprintf(arqs[thread_id], "%d %ld task_create NA %lf %ld\n", omp_get_thread_num(),
            new_task_data->value, omp_get_wtime()-INIT_TIME, ITERATION);
    }
}
```

```

static void on_ompt_callback_task_create(
    omp_data_t *parent_task_data,
    const ompt_frame_t *parent_frame,
    omp_data_t* new_task_data,
    int flag,
    int has_dependences,
    const void *codeptr_ra)
{
    new_task_data->value = next_task();

    if (flag != ompt_task_initial) {
        rst_event_ilsdl(RASTRO_TASK_CREATE, omp_get_thread_num(), new_task_data->value,
            TASKS[ACTUAL_TASK], omp_get_wtime()-INIT_TIME,
            ITERATION);
    } else {
        rst_event_ilsdl(RASTRO_TASK_CREATE, omp_get_thread_num(), new_task_data->value,
            "NA", omp_get_wtime()-INIT_TIME,
            ITERATION);
    }
}

```

Intrusiveness and Scalability of OMPT-Based Tracing Tools for Task-based OpenMP Applications

Methodology

- QR Factorization
- Gauss-Seidel
- Cholesky Factorization

Application parameters

Application	Problem	Block	Tasks Created
QR factorization	2048	64, 128, 192, 258	11440, 1496, 385, 204
Cholesky factorization	2000	50, 80, 100, 200	11480, 2925, 1540, 220
Gauss-Seidel method	10000	250, 500, 1000, 2000	16000, 4000, 1000, 250

Hardware configuration

Nome	CPU	RAM
Cei	2 x Intel(R) Xeon(R) Silver 4116, 2.10 GHz, 48 ths, 24 cores	96 GB DDR4
Hype	2 x Intel(R) Xeon(R) E5-2650 v3, 2.30 GHz, 40 ths, 20 cores	128 GB DDR4
Draco	2 x Intel(R) Xeon(R) E5-2640 v2, 2.00 GHz, 32 ths, 16 cores	64 GB DDR3

- Full factorial (application, block, # threads, etc...)

- Full factorial (application, block, # threads, etc...)
- 10 executions for each combination

- Full factorial (application, block, # threads, etc...)
- 10 executions for each combination
- Metric: Intrusion time (overhead).

Results

Intrusion overview (QR)

object 'df.stats.diff' not found

Intrusion overview (QR)

object 'df.stats.diff' not found

Intrusion overview (QR)

object 'df.stats.diff' not found

Intrusion overview (QR)

object 'df.stats.diff' not found

Intrusion overview (QR)

object 'df.stats.diff' not found

Scalability: Void

object 'df.stats.diff' not found

object 'df.stats.diff' not found

Scalability: OOT

object 'df.stats.diff' not found

object 'df.stats.diff' not found

object 'df.stats.diff' not found

object 'df.stats.diff' not found

Conclusion

- The OMPT interface has a low intrusion by itself (**Void**).

- The OMPT interface has a low intrusion by itself (**Void**).
- The choose of the tracer tool can impact the performance.

- The OMPT interface has a low intrusion by itself (**Void**).
- The choose of the tracer tool can impact the performance.
- Tracing tool choice is crucial depending on the expected result.

Questions?

Contact: rayan.raddatz@inf.ufrgs.br

